

Z-Score Machine Pré-Calculé

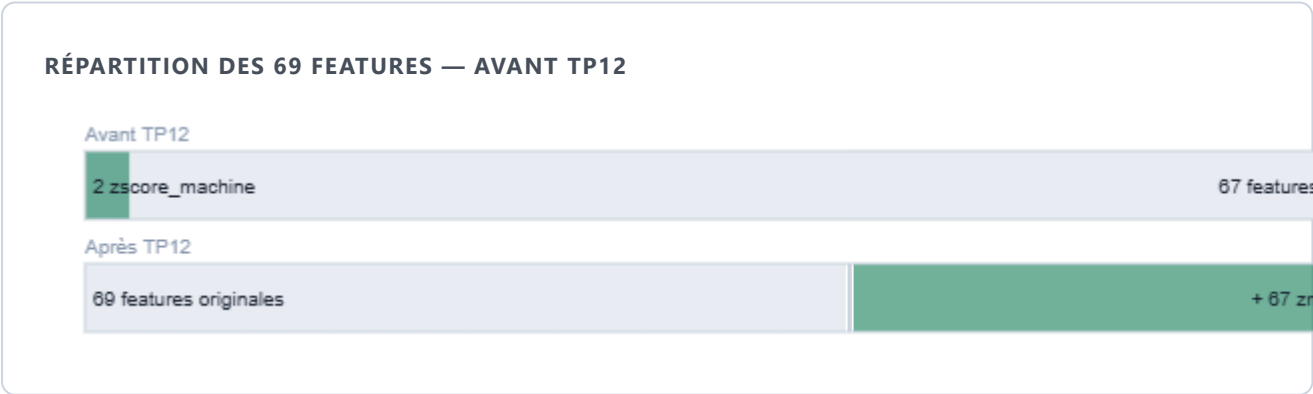
Corriger le domain shift de TP10 en calculant les z-scores machine sur l'historique complet — concept juste, implémentation à affiner.

RÉFÉRENCE	MEILLEUR	TP12 — CE TP
B9-GKF	B11-GKF	B12-GKF
PR-AUC CV0.7792	PR-AUC CV0.7839	PR-AUC CV0.7480
± std0.1772	± std0.1891	± std0.2300
PR-AUC test0.8595	PR-AUC test0.8799	PR-AUC test0.8487
Features69	Features69	Features136
— référence	CV +0.0047 · test +0.0204	CV -0.0312 · test -0.0312

01

Audit des features zscore_machine

Sur 69 features disponibles, seulement **2 features zscore_machine** existaient dans la base. 67 features raw n'avaient aucun équivalent machine-normalisé.



Catégorie	Count	Exemples
zscore_machine existantes	2	temp_zscore_machine, pressure_zscore_machine
Agrégats temporels (1h/6h/24h)	~40	temp_mean_1h, pressure_std_24h, voltage_mean_1h ...
Indicateurs d'incidents	~15	incident_count_prev_24h, incident_max_severity_prev_24h ...
Types d'anomalie	~12	type_blocage_mecanique_count_prev_24h ...

TP12 a pré-calculé un `_zmachine` pour chacune des 67 features raw → feature set augmenté à **136 features**.

02

Concept — Pré-calcul sur historique complet

La correction clé vs TP10 : les μ et σ sont calculés sur la **totalité du dataset**, avant tout split. La machine de validation a toujours ses statistiques propres disponibles.

TP10 VS TP12 — OÙ SONT CALCULÉES LES STATISTIQUES MACHINE ?



```
# TP12 - stats calculées sur df_full (historique complet machine)
def compute_machine_zscores(df_full, raw_cols, suffix="_zmachine"):
    machine_stats = df_full.groupby("machine_id")[raw_cols].agg(["mean", "std"])
    # Toute machine a ses propres  $\mu/\sigma$  - pas de fallback flotte nécessaire
    for col in raw_cols:
        mu = df_full["machine_id"].map(machine_stats[col]["mean"])
        sig = df_full["machine_id"].map(machine_stats[col]["std"]).replace(0, 1)
        result[f"{col}{suffix}"] = (df_full[col] - mu) / sig
```

03

Résultats & Diagnostic

B12-GKF : CV = 0.7480 ± 0.23 — régression vs B11 (0.7839)

Doubler le feature set de 69 → 136 sans re-tuner les HP dégrade les performances. Variance inter-machine en hausse (std 0.19 → 0.23).

Trois hypothèses explicatives

H1 — REDONDANCE

H2 — HP DÉSALIGNÉS

H3 — STACK VS REPLACE

67 paires (raw + zmachine) hautement corrélées. Le signal utile est dilué par des features quasi-identiques dans un référentiel différent.

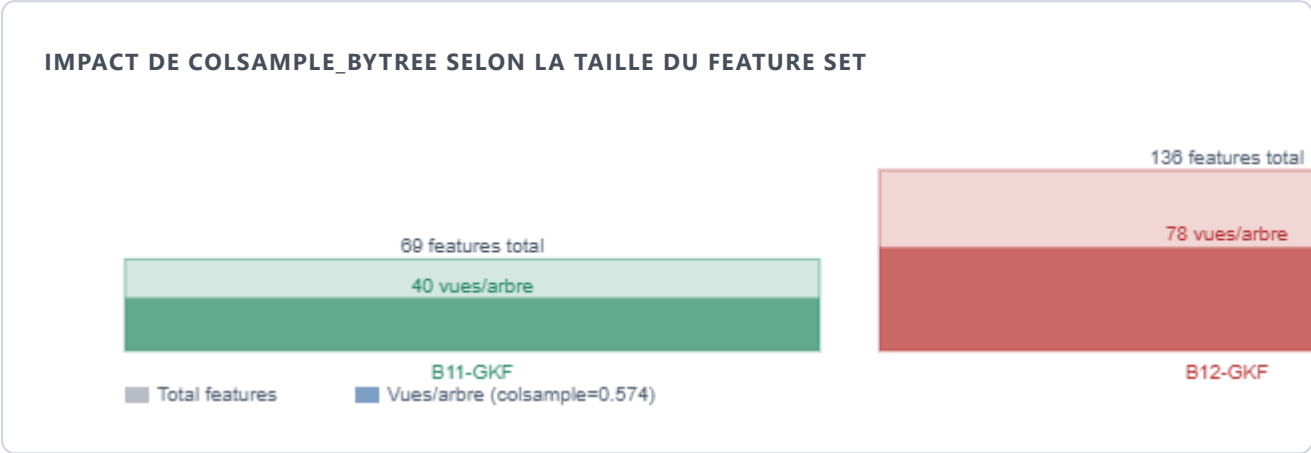
Confirmée

`colsample_bytree=0.574` → $69 \times 0.574 \approx 40$ features/arbre en B11, mais $136 \times 0.574 \approx 78$ en B12. Deux fois plus de features corrélées par arbre.

Confirmée

Empiler zscore sur raw *au lieu de remplacer* crée une redondance inutile. HP B11 optimisés pour 69 features, pas 136.

Confirmée



04

Stratégies — Stack vs Replace vs Select

Stratégie	Features	HP B11 applicables ?	Attente
Stack (TP12 actuel)	136 (raw + zmachine)	Non — colsample_bytree désaligné	Redondance → dégradation
Replace (B12b)	69 (zmachine uniquement)	Oui — même espace	Comparable ou mieux que B11
Optuna 136 (B12c)	136	Re-tuning complet	Potentiellement meilleur
SHAP Select (B12d)	Top-N features	Re-tuning conseillé	Signal concentré

PRIORITÉ 1

B12b — Replace

Garder *uniquement* les zmachine (remplacer les raw). Même N features que B11 → HP réutilisables sans re-tuning.

PRIORITÉ 2

B12c — Optuna 136

Re-tuner Optuna GroupKFold sur le feature set 136. Coût : $150 \text{ fits} \times 2 \times \text{temps}$. HP adaptés à l'espace augmenté.

PRIORITÉ 3

B12d — SHAP Select

Entraîner B12, calculer les importances SHAP sur 136, retenir Top-N. Vérifie si les nouvelles zmachine apportent un signal réel.

Code — Stratégie Replace (B12b)

```
# Identifier les raw qui ont maintenant un équivalent zmachine
zmachine_cols = [c for c in df.columns if "_zmachine" in c]
raw_bases     = {c.replace("_zmachine", "") for c in zmachine_cols}

# Remplacer raw par zmachine – conserver features sans équivalent
FEATURE_COLS_REPLACE = (
    [c for c in FEATURE_COLS if c not in raw_bases] # raw sans zmachine
    + zmachine_cols                                # zmachine
)
# → même N features ≈ 69, HP B11 applicables sans re-tuning
```

05

Règle générale

PRINCIPE RETENU — TP12

Pré-calculer les z-scores machine sur l'historique complet est la bonne approche pour éviter le domain shift de TP10. **Mais empiler les zscore sur les raw sans re-tuner les HP introduit une redondance structurelle.** La stratégie correcte est de remplacer les raw par leurs équivalents zmachine (N features stables → HP réutilisables), ou de re-tuner Optuna sur l'espace augmenté. Doubler le feature set sans adapter `colsample_bytree` revient à exposer deux fois plus de features corrélées par arbre — le modèle ne peut pas en tirer parti.